

TP11 ATELIER : Javascript

Table des matières

1) Problématique.....	1
2) Exercices	1
3) Conclusion.....	19

1) Problématique

Nous cherchons à apprendre un nouveau langage de code : le Javascript.
Afin de pouvoir comprendre comment il marche, comment l'utiliser, son but, et son utilité, nous allons suivre le cours présent sur le site btssiogap.com et effectuer plusieurs exercices qui testeront nos connaissances et notre compréhension du langage.

2) Exercices

1) Dans le premier cours, on nous demande de vérifier qu'il y a bien une erreur dans le fichier **bug.html** en ouvrant la console avec **F12** :

```
✖ Uncaught ReferenceError: lalala is not defined      bug.html:12  
  at bug.html:12:5
```

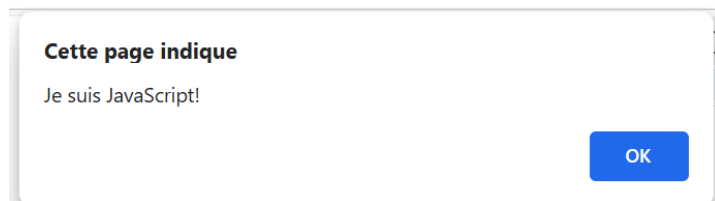
Comme on peut le voir on trouve bien l'erreur.

2) Dans le deuxième cours on nous demande d'afficher un message « Je suis JavaScript ! » à l'aide de la commande **alert**. On nous demande d'abord de le faire directement dans le HTML puis avec un fichier javascript externe.

Afficher une alerte en HTML :

```
<!DOCTYPE html>  
<html>  
  <head>  
    <title>Test JS</title>  
    <script>alert("Je suis JavaScript!")</script>  
  </head>  
  <body>  
  </body>  
</html>
```

Résultat :



Afficher une alerte avec fichier Javascript externe :

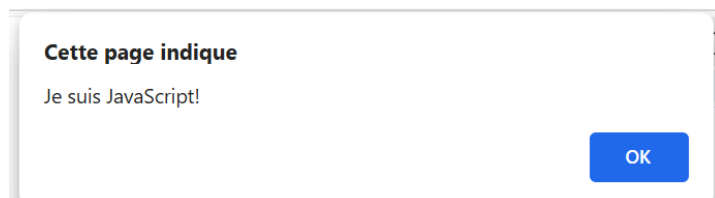
HTML :

```
<!DOCTYPE html>
<html>
<head>
  <title>Test JS</title>
  <script src="alerte.js"></script>
</head>
<body>
</body>
</html>
```

Javascript :

```
alert("Je suis JavaScript!");
```

Résultat :



3) Dans le troisième cours, on apprend comment les variables et les constantes fonctionnent, pour les déclarer il est nécessaire d'utiliser **let** pour les variables et **const** pour les constantes.

On nous demande dans l'exercice de cours de déclarer 2 variables : **admin** et **name**, puis de leur assigner dans l'ordre respectif les valeurs « John » et « name » puis d'afficher la valeur de admin en utilisant la commande **alert**. Ici la subtilité c'est qu'il faut attribuer la valeur de **name** à **admin** sans simplement copier le contenu de **name**. Il faut donc dire à admin de prendre la même valeur que name avec **let admin = name** :

HTML :

```
<!DOCTYPE html>
<html>
<head>
  <title>Test JS</title>
  <script src="alerte.js"></script>
</head>
<body>
</body>
</html>
```

JAVASCRIPT :

```
let name = 'John'
let admin = name
alert(admin)
```

Résultat :

Cette page indique

John

OK

4) Ensuite, on nous demande de créer une variable avec le nom de notre planète et de créer une variable pour stocker le nom du visiteur actuel.

Pour la variable avec le nom de notre planète on l'appellera « **terre** » et pour la variable qui stocke le nom du visiteur actuel on l'appellera « **nomvis** » :

```
let terre
let nomvis
```

5) Puis on nous demande d'analyser un code, puis de voir si il serait juste d'utiliser des majuscules pour birthday et age comme dans cet exemple :

Serait-il juste d'utiliser des majuscules pour birthday ? Pour age ? Ou même pour les deux ?

```
1 const BIRTHDAY = '18.04.1982'; // mettre l'anniversaire en majuscule ?
2
3 const AGE = someCode(BIRTHDAY); // mettre l'âge en majuscule ?
```

Il est conseillé de mettre des constantes qui ne bougeront jamais en majuscule, la constante **birthday** ne changera jamais car il s'agit

simplement de la date d'anniversaire de l'année actuelle, on ne souhaite pas qu'elle change à chaque année. Il est donc conseillé selon la convention de mettre une majuscule pour cette constante. Cependant ce n'est pas pareil pour **AGE**, il serait logique de mettre une majuscule tout simplement par le fait que **age** est une constante, mais on peut voir que cette constante est affectée par une fonction : la fonction **someCode(BIRTHDAY)**. Cela signifie que cette constante change de valeur selon le résultat de la fonction et donc qu'il serait simplement plus logique de transformer cette constante en variable (et donc de ne pas mettre de majuscule au début).

(Point à noter pour moi-même **backticks** : ``` permet d'ajouter une variable ou un calcul dans un **alert** comme avec les virgules en mettant la variable ou le calcul dans `${...}`, booléen = true false ou comparaisons, `typeof` donne le type de l'opération (number,boolean,string, ...)

6) Après cela on nous demande d'indiquer ce que va sortir le script suivant :

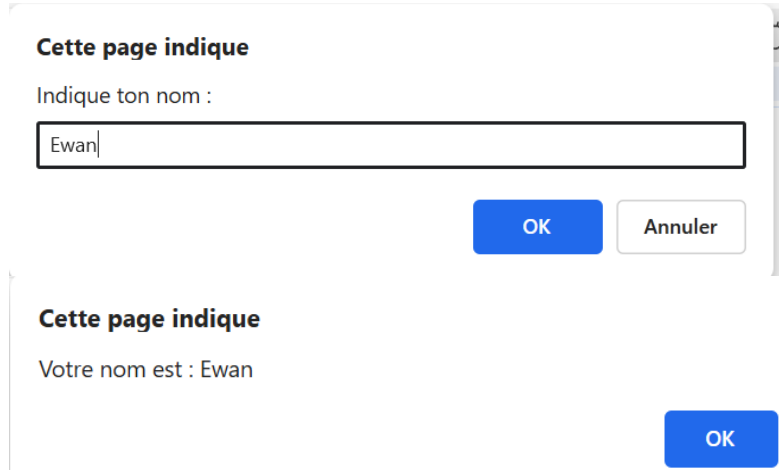
```
1 let name = "Ilya";
2
3 alert( `hello ${1}` ); // ?
4
5 alert( `hello ${"name"}` ); // ?
6
7 alert( `hello ${name}` ); // ?
```

Ce script va d'abord enregistrer la chaîne de caractère « Ilya » dans la variable **name**, puis va créer une pop-up qui va afficher le chiffre 1, puis va créer une deuxième pop-up qui va afficher "name" car le contenu de cet **alert** est entouré de **backticks** qui est prévu généralement à afficher le contenu d'une variable et non d'un **string**. Et le dernier **alert** est celui qui va afficher le contenu de la variable **name** soit **Ilya**.

7) Maintenant on nous demande de faire un code qui demande un nom et l'affiche, voici ce que j'ai fait :

```
let nom = prompt("Indique ton nom :", '');
alert(`Votre nom est : ${nom}`);
```

Voici le résultat :



Cette page indique

Indique ton nom :

OK Annuler

Cette page indique

Votre nom est : Ewan

OK

(Note à moi-même encore une fois, les fonctions Number(xxx), String(xxx) , Boolean(xxx) sont natives)

8) A présent, on veut savoir ce que vaut c et d dans cet exo :

```
1 let a = 1, b = 1;
2
3 let c = ++a; // ?
4 let d = b++; // ?
```

Ici a est incrémenté puis assigné donc, a va devenir 2 puis il va être assigné à c ce qui signifie que c sera égal à 2. Mais pour d, b va être assigné puis incrémenté ce qui veut dire que d va être égal à b puis b va augmenter de 1 donc d = 1 et puis b = 2.

9) Dans cet exercice on veut connaître les valeurs de a et x de ce code :

```
1 let a = 2;
2
3 let x = 1 + (a *= 2);
```

Ici $a = 2$, $x = 1 +$ la résultante de l'opération $a * 2$ ce qui signifie que a va prendre la valeur de $a * 2$ soit $2 * 2$ soit 4 donc $a = 4$ à la fin de code et $x = 1 + 4$ donc 5.

10) Dans celui-ci il faut donner le résultat de chaque expression :

```
1  "" + 1 + 0
2  "" - 1 + 0
3  true + false
4  6 / "3"
5  "2" * "3"
6  4 + 5 + "px"
7  "$" + 4 + 5
8  "4" - 2
9  "4px" - 2
10 "  -9  " + 5
11 "  -9  " - 5
12 null + 1
13 undefined + 1
14 " \t \n" - 2
```

1. Le premier est un string donc ça va forcer 1 et 0 à devenir des strings, donc cela va donner : 10

2. Pour celui-ci il y a l'opérateur – donc “” va se transformer en 0, $-1 + 0 = -1$

3. $true + false = 1 + 0 = 1$

4. L'opérateur / va transformer le string 3 en nombre 3 donc ça va faire $6/3 = 2$

5. De même pour celui-ci, le * va transformer 2 et 3 en nombre donc ça va faire $2 * 3 = 6$

6. + agit différemment des autres opérateurs donc ça simplement rajouter « px » à la fin de l'opération $4 + 5 = 9$ donc $= 9px$

7. Dans le sens inverse si on commence par un string ça va forcer le reste à se transformer en string $= \$45$

8. L'opérateur – force “4” à devenir un nombre donc ça fait $4 - 2 = 2$

9. Soustraire une chaîne de caractère mélangé à un nombre avec un nombre va résulter en erreur = NaN

10. L'opérateur – force “ -9 ” à devenir -9 ce qui va faire $-9+5 = -4$

11. De même ici = -14

12. null se convertit en 0 donc ça fait $0+1=1$

13. undefined est convertit en NaN = $\text{NaN} + 1 = \text{NaN}$ car NaN est contagieux.

14. – force la cdc à se transformer en 0 donc ça fait $0 - 2 = -2$

11) Dans cet exo il faut corriger l'erreur de ce code :

```
1 let a = prompt("First number?", 1);
2 let b = prompt("Second number?", 2);
3
4 alert(a + b); // 12
```

Ici a+b agissent comme une chaîne de caractère, il faut faire en sorte de les convertir en nombre de cette façon :

ajouter des + devant a et b :

`alert(+a + +b)`

12) Pour cet exercice il faut indiquer les résultats des expressions suivantes :

```
1 5 > 4
2 "apple" > "pineapple"
3 "2" > "12"
4 undefined == null
5 undefined === null
6 null == "\n0\n"
7 null === +"\n0\n"
```

1. Cela va afficher true car 5 est plus grand que 4

2. Cela va afficher false car $p > a$
3. comparaison lexicographique : true car 2 est plus grand que le 1 de 12.
4. null et undefined = valeur équivalente = true
5. type différent = false
6. false car null ne se change pas en 0 avec des cdc non vides.
7. Le + convertit « \n0\n » en NaN donc ça demande si null est du même type que NaN et non c'est pas le cas donc : false.

13) On nous demande ici si **alert** sera affiché ou non :

```
1  if ("0") {  
2    alert( 'Hello' );  
3  }
```

La réponse est OUI car la condition est une chaîne de caractère non vide ce qui équivaut à 1 soit true. Donc la boucle marchera en permanence.

14) Ensuite on veut que l'on réécrit if en ? à partir de ce code :

```
1  let result;  
2  
3  if (a + b < 4) {  
4    result = 'Below';  
5  } else {  
6    result = 'Over';  
7  }
```

Voici ce que j'ai fait :

```
let result = (a + b < 4) ? 'Below' : 'Over';
```

15) Pareil ici mais avec if else :


```

1  let message;
2
3  if (login == 'Employee') {
4    message = 'Hello';
5  } else if (login == 'Director') {
6    message = 'Greetings';
7  } else if (login == '') {
8    message = 'No login';
9  } else {
10   message = '';
11 }

```

Mon code :

```

let message = (login == 'Employee') ? 'Hello' :
              (login == 'Director') ? 'Greetings' :
              (login == '') ? 'No login' :
              '';

```

(Pour simplifier je vais répondre à plusieurs exos à la suite) :
 Quel est le résultat de OR ?

Qu'est-ce que le code ci-dessous va sortir ?

```
1 alert( null || 2 || undefined );
```

Quel est le résultat des alertes OR ?

Qu'est-ce que le code ci-dessous va sortir ?

```
1 alert( alert(1) || 2 || alert(3) );
```

Quel est le résultat de AND ?

Qu'est-ce que ce code va afficher ?

```
1 alert( 1 && null && 2 );
```

Quel est le résultat des alertes AND ?

Qu'est-ce que ce code va afficher ?

```
1 alert( alert(1) && alert(2) );
```

16) Le OR retourne la première valeur **truthy** qu'il trouve, ici 2 est la seule valeur truthy donc ça va retourner 2

17) De même ici, sauf qu'ici ça va d'abord effectuer le alert(1) avant d'afficher la valeur truthy donc il va faire apparaitre 1 puis 2 et s'arrêter avant d'afficher 3 (alert1 est undefined donc false donc ça passe à 2)

18) && affiche la première valeur **falsy**, ici la première falsy est null donc ça affichera null

19) Ici && va retourner undefined après avoir affiché 1 car le alert de alert(1) va se transformer en undefined après avoir fait ce code.

Le résultat de OR AND OR

Quel sera le résultat ?

```
1 alert( null || 2 && 3 || 4 );
```

Vérifiez la plage entre

Ecrivez une condition "if" pour vérifier que l'age est compris entre 14 et 90 ans inclus.

"Inclus" signifie que l'age peut atteindre les 14 ou 90 ans.

Vérifiez à l'extérieur de la plage

Ecrivez une condition "if" pour vérifier que l'age n'est PAS compris entre 14 et 90 ans inclus.

Créez deux variantes: la première utilisant NOT !, La seconde – sans ce dernier.

Une question à propos de "if"

Lesquelles de ces alert es vont s'exécuter ?

Quels seront les résultats des expressions à l'intérieur de if (...) ?

20) && est prioritaire, si aucune valeur falsy est trouvé alors la dernière valeur est affiché, ici 2 n'est pas falsy et 3 non plus donc 3 sera affiché.

21)

```
let age = prompt("Quel est votre âge ?", '');
age = Number(age); /

if (age >= 14 && age <= 90) {
  alert('Vous avez entre 14 et 90 ans');
}
```

22)

```
let age = prompt("Quel est votre âge ?", '');
age = Number(age);

if (!(age >= 14 && age <= 90)) {
  alert("L'âge n'est pas compris entre 14 et 90 ans inclus.");
}
```

23) Le premier if retourne -1 car il est truthy et se trouve avant 0, donc alert va s'exécuter

Le deuxième if retourne 0 car il cherche le premier falsy et 0 en est un, donc alert ne va pas s'exécuter

Le troisième if se fait de cette façon : && est prio donc il regarde si il y'a un falsy, -1 est truthy et 1 aussi donc il va retourner le dernier donc 1, puis il va rester (null || 1), ou va chercher le premier truthy possible, null est falsy et 1 truthy donc cela va résulter en un truthy, donc l>alert va s'exécuter.

24) Check the login

```
let user = prompt("Who's there?", '');  
  
if (user === "Admin") {  
    let password = prompt("Password?", '');  
  
    if (password === "TheMaster") {  
        alert("Welcome!");  
    } else if (password === '' || password === null) {  
        alert("Canceled");  
    } else {  
        alert("Wrong password");  
    }  
  
} else if (user === '' || user === null) {  
    alert("Canceled");  
} else {  
    alert("I don't know you");  
-}
```

25) Pour ce code on doit dire ce qu'affiche i à la fin :

```
1  let i = 3;  
2  
3  while (i) {  
4      alert( i-- );  
5  }
```

Ici i affiche 0 car il est montré avant de décrémenter, si ça avait été - i ça aurait montré 1 car ça serait devenu false avant de montrer 0.

26)

Quelles valeurs affiche la boucle while ?

A votre avis, quelles sont les valeurs affichées pour chaque boucle ? Notez-les puis comparez avec la réponse.

Les deux boucles affichent-elles les mêmes valeurs dans l' `alert` ou pas ?

1.

Le préfixe sous forme `++i` :

```
1 let i = 0;
2 while (++i < 5) alert( i );
```

2.

Le postfixe sous forme `i++` :

```
1 let i = 0;
2 while (i++ < 5) alert( i );
```

1. Ici la boucle va afficher 4 car elle va s'arrêter lorsque $5 < 5$ et l'affichage ne va pas se faire car ça incrémente avant d'afficher et ça s'arrête donc avant d'afficher.

2. Et ici, ça va afficher 5 car l'incrémentation se fait après l'affichage donc i prend la valeur 5 après avoir affiché 5.

27)

Quelles valeurs sont affichées par la boucle "for" ?

Pour chaque boucle, notez les valeurs qui vont s'afficher. Ensuite, comparez avec la réponse.

Les deux boucles `alert` les mêmes valeurs ou pas ?

1.

La forme postfix :

```
1 for (let i = 0; i < 5; i++) alert( i );
```

2.

La forme préfix :

```
1 for (let i = 0; i < 5; ++i) alert( i );
```

Ici, alert va afficher la même valeur car l'affichage se fait pas directement après la fin de l'incrémentation ça va d'abord effectuer l'incrémentation et l'affichage puis afficher le résultat.

28)

Extraire les nombres pairs dans la boucle

Utilisez la boucle `for` pour afficher les nombres pairs de 2 à 10.

Voici mon code :

```
for (let i = 2; i <=10; i++) {  
  if (i % 2 == 0) {  
    alert( i );  
  }  
}
```

29)

Remplacer "for" par "while"

Réécrivez le code en modifiant la boucle `for` en `while` sans modifier son comportement (la sortie doit rester la même).

```
1 for (let i = 0; i < 3; i++) {  
2   alert( `number ${i}!` );  
3 }
```

```
let i = 0;  
while (i<3) {  
  alert( `number ${i}!` );  
}
```

30)

Répéter jusqu'à ce que l'entrée soit correcte

Ecrivez une boucle qui demande un nombre supérieur à 100. Si le visiteur saisit un autre numéro, demandez-lui de le saisir à nouveau.

La boucle doit demander un numéro jusqu'à ce que le visiteur saisisse un nombre supérieur à 100 ou annule l'entrée/entre une ligne vide.

Ici, nous pouvons supposer que le visiteur ne saisit que des chiffres. Il n'est pas nécessaire de mettre en œuvre un traitement spécial pour une entrée non numérique dans cette tâche.

```
let num;

do {
  // Le résultat de prompt est directement converti en nombre
  num = Number(prompt("Entrez un nombre > 100 :", " "));
} while (num <= 100);
```

31)

Extraire des nombres premiers

Un nombre entier supérieur à 1 est appelé un **Nombre premier** s'il ne peut être divisé sans reste par rien d'autre que 1 et lui-même.

En d'autres termes, $n > 1$ est un nombre premier s'il ne peut être divisé de manière égale par autre chose que 1 et n .

Par exemple, 5 est un nombre premier, car il ne peut pas être divisé sans reste par 2, 3 et 4.

Écrivez un code qui produit les nombres premiers dans l'intervalle 2 à n .

Pour $n = 10$, le résultat sera 2, 3, 5, 7.

P.S. Le code devrait fonctionner pour n'importe quel n et aucune valeur fixe ne doit être codé en dur.

```
let nbpremier = 10;

NombrePremier : // On utilise une ancre pour la condition du if
for (let i = 2; i <= n; i++) {
  for (let diviseur = 2; diviseur < i; diviseur++) {
    if (i % diviseur == 0) continue NombrePremier; // continue et affiche si nb est premier
  }
  alert(i);
}
```

32)

Réécrire le "switch" dans un "if"

Écrivez le code en utilisant `if..else` qui correspondrait au `switch` suivant :

```
1 switch (browser) {
2   case 'Edge':
3     alert( "You've got the Edge!" );
4     break;
5
6   case 'Chrome':
7   case 'Firefox':
8   case 'Safari':
9   case 'Opera':
10    alert( 'Okay we support these browsers too' );
11    break;
12
13  default:
14    alert( 'We hope that this page looks ok!' );
15 }
```

```
if (browser == 'Edge') {
  alert("You've got the Edge!");
} else if (browser == 'Chrome' ||
  browser == 'Firefox' ||
  browser == 'Safari' ||
  browser == 'Opera') {
  alert('Okay we support these browsers too');
} else {
  alert('We hope that this page looks ok!');
}
```

33)

Réécrire le "if" dans un "switch"

Réécrivez le code ci-dessous en utilisant une seule instruction `switch` :

```
1 let a = +prompt('a?', '');
2
3 if (a == 0) {
4   alert( 0 );
5 }
6 if (a == 1) {
7   alert( 1 );
8 }
9
10 if (a == 2 || a == 3) {
11   alert( '2,3' );
12 }
```

```

let a = +prompt('a?', '');

switch (a) {
  case 0:
    alert(0);
    break;

  case 1:
    alert(1);
    break;

  case 2:
  case 3:
    alert('2,3');
    break;
}

```

34)

Est-ce que "else" est requis ?

La fonction suivante renvoie `true` si le paramètre `age` est supérieur à 18.

Sinon, il demande une confirmation et renvoie son résultat :

```

1 function checkAge(age) {
2   if (age > 18) {
3     return true;
4   } else {
5     // ...
6     return confirm('Did parents allow you?');
7   }
8 }

```

La fonction fonctionnera-t-elle différemment si `else` est supprimé ?

```

1 function checkAge(age) {
2   if (age > 18) {
3     return true;
4   }
5   // ...
6   return confirm('Did parents allow you?');
7 }

```

Existe-t-il une différence dans le comportement de ces deux variantes ?

Il n'y a aucune différence, dans les deux cas le `return confirm('Did parents allow you?')` s'exécute quand la condition `if` est fausse

35)

Réécrivez la fonction en utilisant '?' ou '||'

La fonction suivante renvoie `true` si le paramètre `age` est supérieur à 18.

Sinon, il demande une confirmation et renvoie le résultat.

```
1 function checkAge(age) {
2   if (age > 18) {
3     return true;
4   } else {
5     return confirm('Did parents allow you?');
6   }
7 }
```

Réécrivez-le, pour effectuer la même chose, mais sans `if`, et en une seule ligne.

Faites deux variantes de `checkAge` :

1. En utilisant un opérateur point d'interrogation `?`
2. En utilisant OU `||`

```
function checkAge(age) {
  return (age > 18) ? true : confirm('Did parents allow you?');
}
```

```
function checkAge(age) {
  return (age > 18) || confirm('Did parents allow you?');
}
```

36)

Fonction min(a, b)

Ecrivez une fonction `min(a, b)` qui renvoie le plus petit des deux nombres `a` et `b`.

Par exemple :

```
1 min(2, 5) == 2
2 min(3, -1) == -1
3 min(1, 1) == 1
```

```
function min(a, b) {
  if (a < b) {
    return a;
  } else {
    return b;
  }
}
```

37)

Fonction pow(x,n)

Ecrivez une fonction `pow(x, n)` qui renvoie `x` à la puissance `n`. Ou, autrement dit, multiplie `x` par lui-même `n` fois et renvoie le résultat.

```
1 pow(3, 2) = 3 * 3 = 9
2 pow(3, 3) = 3 * 3 * 3 = 27
3 pow(1, 100) = 1 * 1 * ... * 1 = 1
```

Créez une page Web qui demande (prompt) `x` et `n`, puis affiche le résultat de `pow(x, n)`.

P.S. Dans cette tâche, la fonction ne doit prendre en charge que les valeurs naturelles de `n` : entiers supérieurs à 1.

```
function pow(x, n) {
  let resultat = x;

  for (let i = 1; i < n; i++) {
    resultat = resultat*x;
  }

  return resultat;
}

let x = prompt("Donnez x", '');
let n = prompt("Donnez n", '');

if (n < 1) {
  alert(`La puissance ${n} ne peut pas être utilisé, utilisez une
  puissance plus grande ou égale à 1`);
} else {
  alert( pow(x, n) );
}
```

38)

Réécrire avec les fonctions fléchées

Remplacez les expressions de fonction par des fonctions fléchées dans le code ci-dessous :

```
1 function ask(question, yes, no) {
2   if (confirm(question)) yes();
3   else no();
4 }
5
6 ask(
7   "Do you agree?",
8   function() { alert("You agreed."); },
9   function() { alert("You canceled the execution."); }
10 );
```

```
function ask(question, yes, no) {  
  if (confirm(question)) yes();  
  else no();  
}  
  
ask(  
  "Do you agree?",  
  () => alert("You agreed."),  
  () => alert("You canceled the execution.")  
);
```

3) Conclusion

En conclusion j'ai appris durant TP à répondre à plusieurs exercices en utilisant les cours sur le JavaScript, cela m'a permis de comprendre qu'il existe différentes façons de coder pour produire un même résultat et j'ai compris l'importance qu'avait ce langage dans le monde de la programmation.